Andrew Yuan
ECE357 PS5

1) First the kernel modifies the %cr3 register with the physical address of the PGD for the new process. .If the CPU doesn't have the PCID feature, the CPU hardware flushes all entries in the TLB (cache) without the 'Global' flag, meaning all User-Mode PTE's get flushed. Since they're no longer cached, the page-table must rewalk the page table structure and load the correct translation.
   If the CPU does have the PCID feature, each VA for each process gets tagged with a unique ID. Since each PTE now has an associated PCID, when the kernel modifies the %cr3 register along with the new PCID, the TLB does not need to be flushed since the hardware can distinguish which process each translation belongs to.

   b) The initial stack region occupies 0xBFFFFFFF → 0xBFFFF000 (since the stack grows downwards). The faulting address is 0xBFFFEFFC which is lower than 0xBFFFF000 but since this is "close" to the stack pointer, the kernel interprets the fault as a stack growth rather than an invalid access. The stack region is then expanded by one page "down" as if it was done by the mremap system call. This is assuming the memory region just above the faulted address has the GROWSDOWN property. Note that now the program continues as if the page fault occurred in a valid memory region.

   c) Since the CounterStrike executable is loaded on a remote server, when we play the game, only the portions of the code that we play with get loaded into memory via demand paging, while the rest stay on the remote server. When we encounter a new feature and we need to load in new code, we incur a page fault since that code is not on disk. As the kernel tries to resolve this page fault, it realizes the connection to the remote server is broken thus it delivers a SIGBUS error to the process.

   d) If /home/student/a.out is being executed as process 6767 and process 555 tries to open in Read/Write mode, that operation will fail and receive an error ETXTBUSY. This is because the MAP_DENYWRITE flag is set in the text and data regions of the /home/student/a.out executable, which disallows any modification of the original file during execution. This prevents possible corruption since any modification to the underlying file could lead to mixed old/new instructions being paged in.

   e) In 64-bit X86 Architecture, each entry of the PGD occupies 512 (2^39) GB and since the first PDE is all-0 (meaning the virtual memory is unmapped in this region), we have 0x0000 0000 0000 0000 to (0x0000 0080 0000 0000 - 1) = **0x0000 0007F FFFF FFFF.**

First Free page number = 0x00100000 ascending order ↑

text: 0x08040000

data: 0x08041000

bss: 0x08042000

stack: 0xBFFFF000 → 0xBFFFFFFF

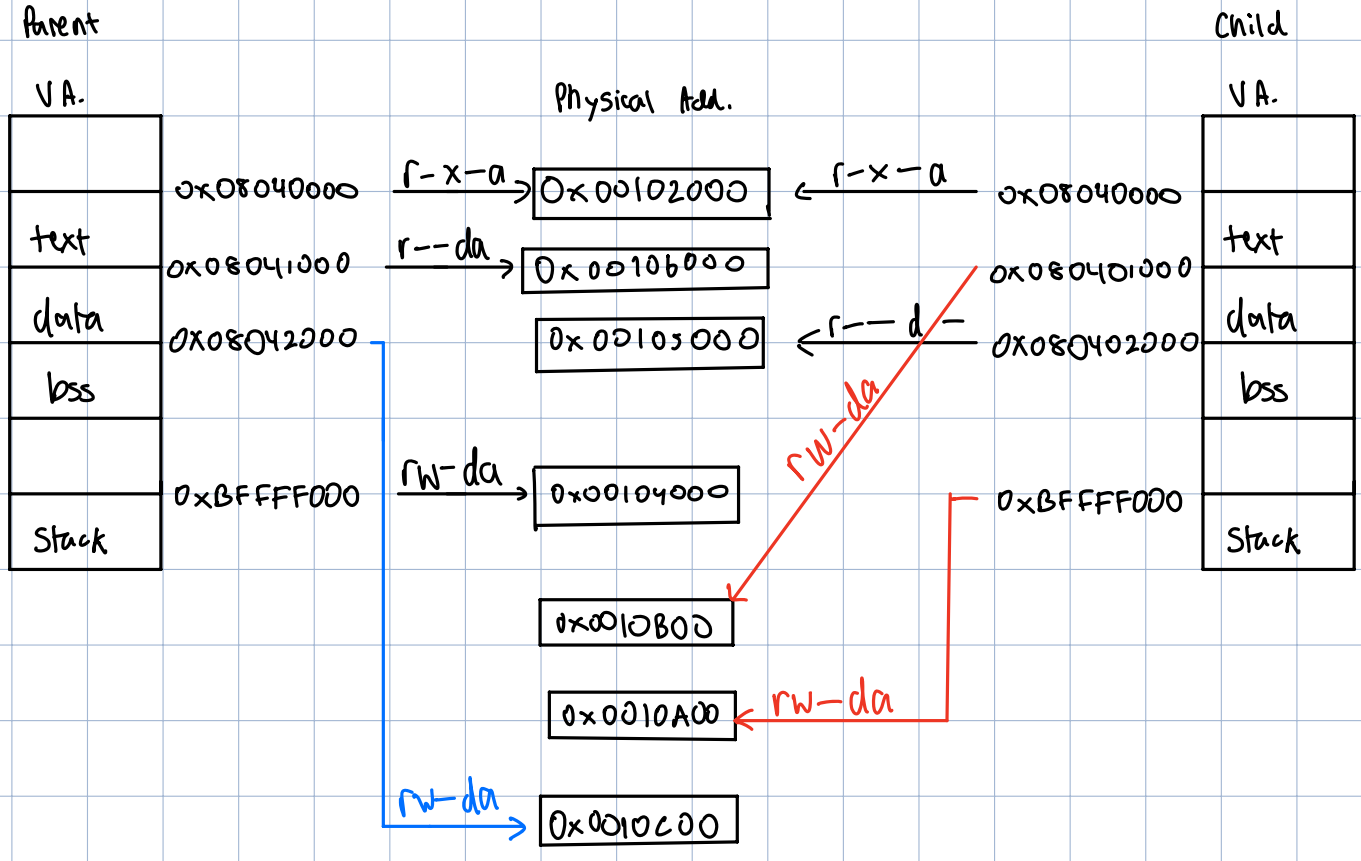
Not shown:

Parent PGD: 100 Child PGD: 107

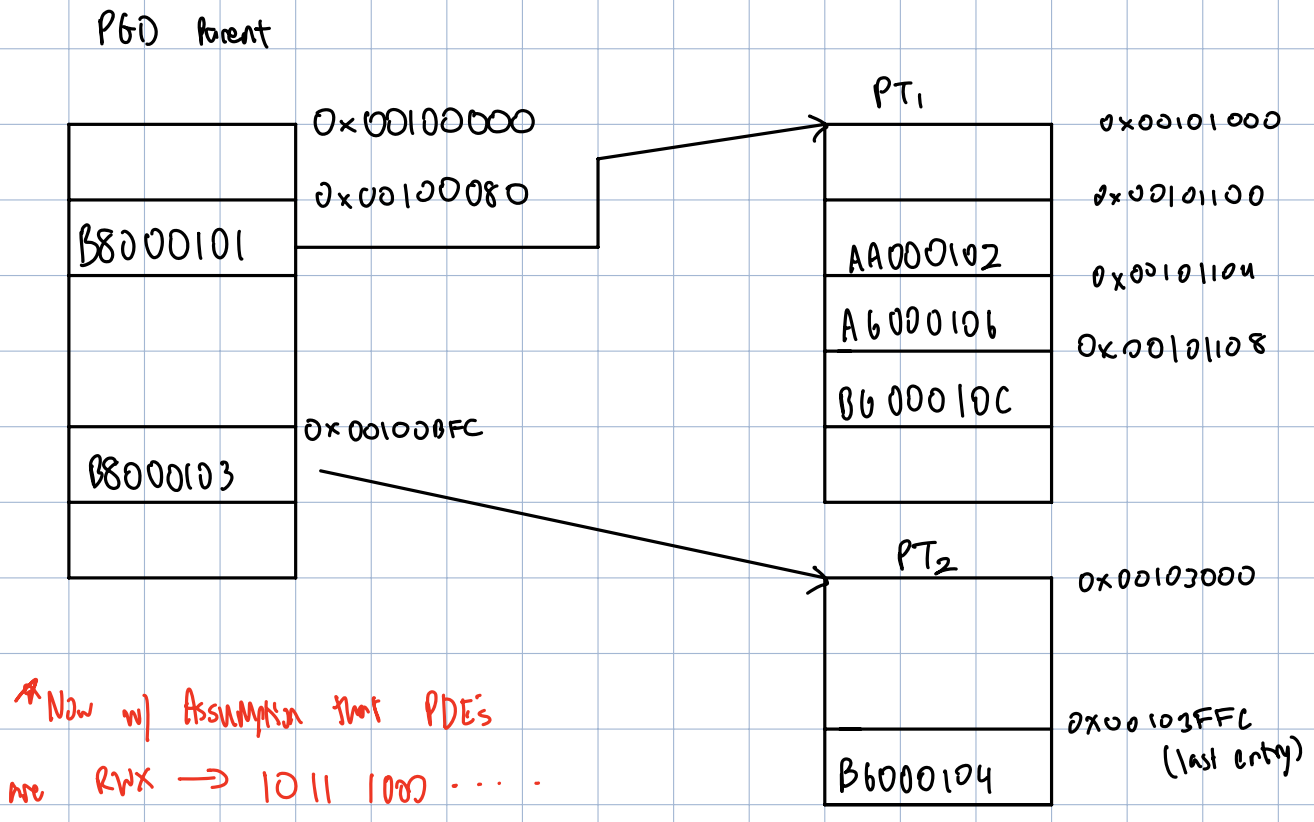
PT_1: 101 —copied→ PT_3: 108

PT_2: 103 —copied→ PT_4: 109

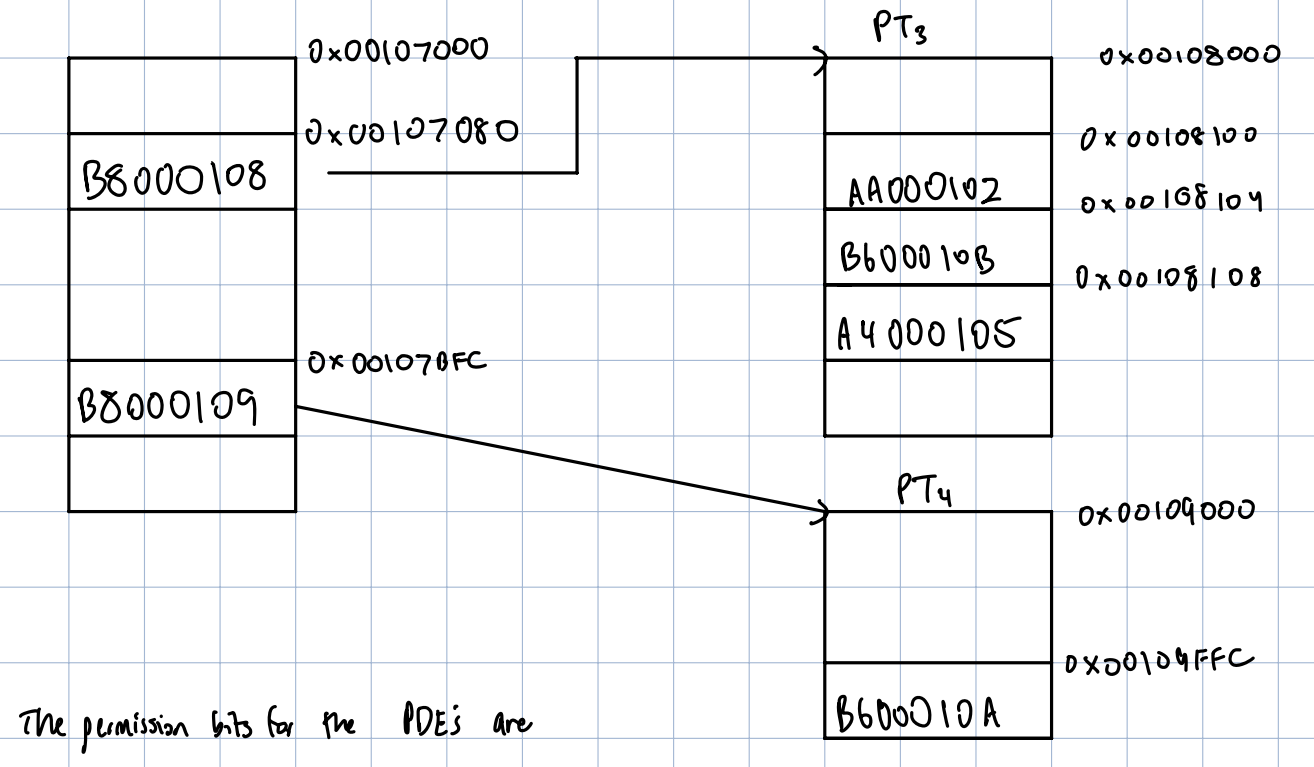
Conceptual Model



Physical Model



*Now w/ assumption that PDE's are RWX → 1011 1000 . . . .
B 8



The permission bits for the PDE's are confusing me !!